Practical No 4.2

Doubly Linked List(Train Reservation System)

Name: Arjun Mankuskar

PRN:202401110038

Branch: CSE (AIML)

Batch: A2

```
//CODE
#include <iostream>
#include <string>

using namespace std;



struct Node {
    string passengerName;
    string trainNumber;
    int seatNumber;
    Node* prev;
    Node* next;



    Node(string name, string train, int seat) {
        passengerName = name;
        trainNumber = train;
        seatNumber = seat;
        prev = NULL;
        next = NULL;
    }
};

Node* head = NULL;

void insertAtBeginning(string name, string train, int seat) {
    Node* newNode = new Node(name, train, seat);
    if (head == NULL) {
        head = newNode;
    } else {
```

```
        newNode->next = head;
        head->prev = newNode;
        head = newNode;
    }
}

void insertAtLast(string name, string train, int seat) {
    Node* newNode = new Node(name, train, seat);
    if (head == NULL) {
        head = newNode;
        return;
    }
    Node* temp = head;
    while (temp->next != NULL) {
        temp = temp->next;
    }
    temp->next = newNode;
    newNode->prev = temp;
}

void deleteFromPosition(int pos) {
    if (head == NULL) {
        cout << "List is empty!\n";
        return;
    }
    if (pos < 1) {
        cout << "Invalid position!\n";
        return;
    }

    Node* temp = head;
```

```
    if (pos == 1) {
        head = head->next;
        if (head != NULL) {
            head->prev = NULL;
        }
        delete temp;
        cout << "First reservation deleted.\n";
        return;
    }

    int count = 1;
    while (temp != NULL && count < pos) {
        temp = temp->next;
        count++;
    }

    if (temp == NULL) {
        cout << "Position out of range!\n";
        return;
    }

    if (temp->prev != NULL) {
        temp->prev->next = temp->next;
    }
    if (temp->next != NULL) {
        temp->next->prev = temp->prev;
    }
    delete temp;
    cout << "Reservation at position " << pos << " deleted.\n";
}
```

```
void sortList() {
    if (head == NULL || head->next == NULL) {
        cout << "List is too small to sort.\n";
        return;
    }

    for (Node* i = head; i->next != NULL; i = i->next) {
        for (Node* j = i->next; j != NULL; j = j->next) {
            if (i->seatNumber > j->seatNumber) {
                string tempName = i->passengerName;
                string tempTrain = i->trainNumber;
                int tempSeat = i->seatNumber;

                i->passengerName = j->passengerName;
                i->trainNumber = j->trainNumber;
                i->seatNumber = j->seatNumber;

                j->passengerName = tempName;
                j->trainNumber = tempTrain;
                j->seatNumber = tempSeat;
            }
        }
    }
    cout << "List sorted by seat number.\n";
}

void display() {
    if (head == NULL) {
        cout << "List is empty! No reservations to show.\n";
        return;
    }
```

```
    Node* temp = head;
    cout << "\n--- Current Reservations ---\n";
    int count = 1;
    while (temp != NULL) {
        cout << count << ". ";
        cout << "Passenger: " << temp->passengerName;
        cout << ", Train: " << temp->trainNumber;
        cout << ", Seat: " << temp->seatNumber << endl;
        temp = temp->next;
        count++;
    }
}

int main() {
    int choice, seat, pos;
    string name, train;

    do {
        cout << "\n--- Train Reservation System ---\n";
        cout << "1. Add Reservation at Beginning\n";
        cout << "2. Add Reservation at End\n";
        cout << "3. Delete a Reservation by Position\n";
        cout << "4. Sort Reservations by Seat Number\n";
        cout << "5. Display All Reservations\n";
        cout << "6. Exit\n";
        cout << "Enter choice: ";
        cin >> choice;

        switch (choice) {
            case 1:
```

```
            cout << "Enter Passenger Name: ";
            cin.ignore();
            getline(cin, name);
            cout << "Enter Train Number: ";
            getline(cin, train);
            cout << "Enter Seat Number: ";
            cin >> seat;
            insertAtBeginning(name, train, seat);
            cout << "Reservation added.\n";
            break;
        case 2:
            cout << "Enter Passenger Name: ";
            cin.ignore();
            getline(cin, name);
            cout << "Enter Train Number: ";
            getline(cin, train);
            cout << "Enter Seat Number: ";
            cin >> seat;
            insertAtLast(name, train, seat);
            cout << "Reservation added.\n";
            break;
        case 3:
            cout << "Enter position to delete: ";
            cin >> pos;
            deleteFromPosition(pos);
            break;
        case 4:
            sortList();
            break;
        case 5:
            display();
```

```
            break;
        case 6:
            cout << "Exiting program.\n";
            break;
        default:
            cout << "Invalid choice! Try again.\n";
      }
   } while (choice != 6);

   return 0;
}
```

```
/*

--- Train Reservation System ---
1. Add Reservation at Beginning
2. Add Reservation at End
3. Delete a Reservation by Position
4. Sort Reservations by Seat Number
5. Display All Reservations
6. Exit
Enter choice: 1
Enter Passenger Name: a
Enter Train Number: 5
Enter Seat Number: 45
Reservation added.

--- Train Reservation System ---
1. Add Reservation at Beginning
```

2. Add Reservation at End

3. Delete a Reservation by Position

4. Sort Reservations by Seat Number

5. Display All Reservations

6. Exit

Enter choice: 2

Enter Passenger Name: s

Enter Train Number: 8

Enter Seat Number: 12

Reservation added.

--- Train Reservation System ---

1. Add Reservation at Beginning

2. Add Reservation at End

3. Delete a Reservation by Position

4. Sort Reservations by Seat Number

5. Display All Reservations

6. Exit

Enter choice: 4

List sorted by seat number.

--- Train Reservation System ---

1. Add Reservation at Beginning

2. Add Reservation at End

3. Delete a Reservation by Position

4. Sort Reservations by Seat Number

5. Display All Reservations

6. Exit

Enter choice: 5

--- Current Reservations ---

1. Passenger: s, Train: 8, Seat: 12

2. Passenger: a, Train: 5, Seat: 45

--- Train Reservation System ---

1. Add Reservation at Beginning

2. Add Reservation at End

3. Delete a Reservation by Position

4. Sort Reservations by Seat Number

5. Display All Reservations

6. Exit

Enter choice: 3

Enter position to delete: 2

Reservation at position 2 deleted.

--- Train Reservation System ---

1. Add Reservation at Beginning

2. Add Reservation at End

3. Delete a Reservation by Position

4. Sort Reservations by Seat Number

5. Display All Reservations

6. Exit

Enter choice: 5

--- Current Reservations ---

1. Passenger: s, Train: 8, Seat: 12

--- Train Reservation System ---

1. Add Reservation at Beginning

2. Add Reservation at End

3. Delete a Reservation by Position

4. Sort Reservations by Seat Number

5. Display All Reservations

6. Exit

Enter choice: 2

Enter Passenger Name: j

Enter Train Number: 6

Enter Seat Number: 75

Reservation added.

--- Train Reservation System ---

1. Add Reservation at Beginning

2. Add Reservation at End

3. Delete a Reservation by Position

4. Sort Reservations by Seat Number

5. Display All Reservations

6. Exit

Enter choice: 5

--- Current Reservations ---

1. Passenger: s, Train: 8, Seat: 12

2. Passenger: j, Train: 6, Seat: 75

--- Train Reservation System ---

1. Add Reservation at Beginning

2. Add Reservation at End

3. Delete a Reservation by Position

4. Sort Reservations by Seat Number

5. Display All Reservations

6. Exit

Enter choice: 1

Enter Passenger Name: p

Enter Train Number: 6

Enter Seat Number: 18

Reservation added.

--- Train Reservation System ---

1. Add Reservation at Beginning

2. Add Reservation at End

3. Delete a Reservation by Position

4. Sort Reservations by Seat Number

5. Display All Reservations

6. Exit

Enter choice: 5

--- Current Reservations ---

1. Passenger: p, Train: 6, Seat: 18

2. Passenger: s, Train: 8, Seat: 12

3. Passenger: j, Train: 6, Seat: 75

--- Train Reservation System ---

1. Add Reservation at Beginning

2. Add Reservation at End

3. Delete a Reservation by Position

4. Sort Reservations by Seat Number

5. Display All Reservations

6. Exit

Enter choice: 6

*/